



Brock University
Faculty of Mathematics & Science
Department of Computer Science

COSC 3P93: Parallel Computing
4P78 Project Step 3

Prepared By:

Parker TenBroeck 7376726
pt21zs@brocku.ca

Instructor:
Robson De Grande

December 12, 2025

1 Intro

My topic is rendering of triangle meshes.

This topic is explored in great depth and has a wide array of different techniques and optimizations to produce varying levels of realism. Additionally parallelization often goes hand in hand when talking about these techniques as they are designed for GPUs.

I heavily relied on material and techniques talked about on the Learn OpenGL [4] website.

The final rendered results for all three scenes (1080) can be seen here [demo](#)

2 Algorithm Overview

There are a 4 major steps involved in this process

1. Transforming from local space to clip space through the Projection, View, and Model matrices
2. Clipping out of view verticies and triangles.
3. Rasterizing triangles to the frame buffer
4. Performing per fragment(pixel) lighting

2.1 Projection

the projection matrix translates coordinates into clip space (4).

fov = the field of view for our camera

far = the farthest point our camera can see

$near$ = the closest point our camera can see

$$M_{projection} = \begin{bmatrix} \frac{1}{aspect * \tan(\frac{fov}{2})} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\frac{fov}{2})} & 0 & 0 \\ 0 & 0 & -\frac{far+near}{far-near} & -\frac{2 \times far \times near}{far-near} \\ 0 & 0 & -1 & 0 \end{bmatrix} \quad (1)$$

2.2 View

The view matrix transforms world space coordinates into view space coordinates where x is left and right, y is up and down, and z is depth.

P_{target} = Where the camera is looking

P_{eye} = Where the camera is

$\overrightarrow{V_{world\ up}}$ = the up direction of the world $\langle 0 \ 1 \ 0 \rangle$

$\overrightarrow{V_{cam\ forward}}$ = $\langle P_{target} - P_{eye} \rangle$

$\overrightarrow{V_{cam\ right}}$ = $\langle \overrightarrow{V_{cam\ forward}} \times \overrightarrow{V_{world\ up}} \rangle$

$\overrightarrow{V_{cam\ up}}$ = $\langle \overrightarrow{V_{cam\ right}} \times \overrightarrow{V_{cam\ forward}} \rangle$

$$M_{view} = \begin{bmatrix} \overrightarrow{V_{cam\ right}x} & \overrightarrow{V_{cam\ right}y} & \overrightarrow{V_{cam\ right}z} & \overrightarrow{V_{cam\ right}} \cdot P_{eye} \\ \overrightarrow{V_{cam\ up}x} & \overrightarrow{V_{cam\ up}y} & \overrightarrow{V_{cam\ up}z} & \overrightarrow{V_{cam\ up}} \cdot P_{eye} \\ -\overrightarrow{V_{cam\ forward}x} & -\overrightarrow{V_{cam\ forward}y} & -\overrightarrow{V_{cam\ forward}z} & \overrightarrow{V_{cam\ forward}} \cdot P_{eye} \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2)$$

2.3 Model

The model matrix transforms coordinates from their local model positions to world space.

$$\begin{aligned}
P_{pos} &= \text{world space location of the object} \\
\overrightarrow{V_{scale}} &= x, y, z \text{ scale of object} \\
A_{rotation} &= \text{euler angles of objects rotation, raw,pitch,roll} \\
M_{yaw} &= \begin{bmatrix} \cos(yaw) & -\sin(yaw) & 0 \\ \sin(yaw) & \cos(yaw) & 0 \\ 0 & 0 & 1 \end{bmatrix} \\
M_{pitch} &= \begin{bmatrix} \cos pitch & 0 & \sin(pitch) \\ 0 & 1 & 0 \\ -\sin pitch & 0 & \cos pitch \end{bmatrix} \\
M_{roll} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(roll) & -\sin(roll) \\ 0 & \sin(roll) & \cos(roll) \end{bmatrix} \\
M_{rot} &= M_{yaw} \cdot M_{pitch} \cdot M_{roll} \\
M_{scale} &= \begin{bmatrix} \overrightarrow{V_{scalex}} & 0 & 0 \\ 0 & \overrightarrow{V_{scaley}} & 0 \\ 0 & 0 & \overrightarrow{V_{scalez}} \end{bmatrix} \\
M_{rot\ scale} &= M_{rot} \cdot M_{scale} \\
M_{model} &= \begin{bmatrix} M_{rot\ scale11} & M_{rot\ scale12} & M_{rot\ scale13} & P_{pos_x} \\ M_{rot\ scale21} & M_{rot\ scale22} & M_{rot\ scale23} & P_{pos_y} \\ M_{rot\ scale31} & M_{rot\ scale32} & M_{rot\ scale33} & P_{pos_z} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{3}
\end{aligned}$$

2.4 Clip

$$\begin{aligned}
V_{world} &= M_{model} \cdot V_{local} \\
V_{clip} &= M_{projection} \cdot M_{view} \cdot M_{model} \cdot V_{local} \\
M_{normal} &= M_{model}^{-1T} \\
\langle N_{world} \rangle &= M_{normal} \cdot \langle N_{local} \rangle
\end{aligned}$$

If the resulting vertex is outside this range it is not visible and can be clipped.

$$\begin{aligned}
-V_{clip_w} &\leq V_{clip_x} \leq V_{clip_w} \\
-V_{clip_w} &\leq V_{clip_y} \leq V_{clip_w} \\
-V_{clip_w} &\leq V_{clip_z} \leq V_{clip_w}
\end{aligned} \tag{4}$$

2.5 Depth Perspective

After clipping each vertexes x, y, z components are divided by their w component to give depth perspective.

2.6 Screen Space

After depth perspective the x, y components are shifted over by $\frac{1}{2}$ then scaled to the screen width and height.

2.7 Rasterization

The rasterizer first finds a bounding box around the screen space coordinates. Then iterates over each pixel in the box finding the barycentric coordinates at that pixel. It uses these to determine if the point is actually in the triangle or not. and if it is find the linearly interpolated world position, uv, normal, tangent, and bitangent at that point from the three points defining the triangle.

2.7.1 Perspective Correction

World position, uv, normal, tangent, and bitangent all need to be perspective corrected when being linearly interpolated during rasterization. This is done by scaling each vector type by $\frac{1}{w}$, doing the linear interpolation between face points. then scaling the resulting interpolated vector again by a linearly interpolated $\frac{1}{w}$.

2.8 Fragment

The fragment uses Binn-Phong shading

$$\begin{aligned}
 \langle \overrightarrow{V_{light\ dir}} \rangle &= \langle P_{light} - P_{fragment} \rangle \\
 \text{light power} &= \frac{\text{light intensity}}{|P_{light} - P_{fragment}|^2} \\
 \text{lambertian} &= \max(0, \langle \overrightarrow{V_{light\ dir}} \rangle \cdot \langle N_{fragment} \rangle) \\
 \langle \overrightarrow{V_{view\ dir}} \rangle &= \langle P_{eye} - P_{fragment} \rangle \\
 \langle \overrightarrow{V_{half\ dir}} \rangle &= \langle \langle \overrightarrow{V_{view\ dir}} \rangle + \langle \overrightarrow{V_{light\ dir}} \rangle \rangle \\
 \text{specular} &= \max(0, \langle \overrightarrow{V_{half\ dir}} \rangle \cdot \langle N_{fragment} \rangle)^{\text{fragment shininess}} \\
 C_{\text{spec}} &= C_{\text{light}} \times \text{specular} \times \text{light power} \\
 C_{\text{diff}} &= C_{\text{light}} \times \text{lambertian} \times \text{light power}
 \end{aligned}$$

3 Parallel Design

3.1 OpenMP

Each fragment(pixel) has no dependencies and each fragment performs the same computation so we can statically split the frame up and run it in parallel.

```

1 void fragment(FrameBuffer& frame, const Scene& scene, const ResourceStore& resources) {
2     #pragma omp parallel for schedule(static)
3     for (usize i = 0; i < frame.size(); i++) {
4         frame[i] = frame[i].fragment_shader(scene, resources);
5     }
6 }

```

The projection for each face also has no dependencies on eachother, the only step is rasterization which can potentially have multiple threads writing to the same pixel at the same time.

```

1 void render_mesh(FrameBuffer& frame, const Mesh& mesh) {
2     #pragma omp parallel for schedule(static)
3     for (usize i = 0; i < mesh.faces.size(); ++i) {
4         Face face = mesh.faces[i];
5         project_rasterize_face(frame, face);
6     }
7 }

```

Special consideration, setting pixels from many different threads in the same frame buffer can lead to race conditions and improper pixel writes.

The naive solution would be to put a critical zone in the region where we check if the current pixels depth is farther than the new pixels depth. The issue with this is in the vast majority of the time we don't need this critical section. This operation happens millions of times per second so finding a better solution will be hugely beneficial.

What if we have a per pixel lock? this solution is better but using a traditional pthread mutex would waste a significant amount of time potentially putting an OS thread to sleep if it doesn't immediately acquire it and wastes a ton of space per pixel to store the mutex itself.

Per pixel spin lock? This is better as we know we will only be locking for a single comparison and few bytes written to memory so will only spin for a minimal number of cycles. but we still need extra space for the lock value.

Using values already in our pixel. If we use the depth value in our pixel (which can only go from 0x0 to 0xFFFFFFFF) we can use a special value 0xFFFFFFFF that signals "this pixel is locked". This is the final idea which led to the code below

```
1 void set_smaller_depth(Pixel pixel) {
2     constexpr u32 LOCK_VALUE = 0xFFFFFFFF;
3
4     const std::atomic<u32>* atomic = &this->depth;
5     u32 depth;
6     while ((depth = atomic->exchange(LOCK_VALUE, std::memory_order_acquire)) == LOCK_VALUE){}
7     if (depth > pixel.depth) {
8         depth = pixel.depth;
9         std::memcpy(this, &pixel, sizeof(Pixel)-sizeof(pixel.depth));
10    }
11    atomic->store(depth, std::memory_order_release);
12 }
```

This works! it has very little to no noticeable performance impact (less than 1% difference between runs using OpenMP with a single thread and the serial version). Does not require extra data stored per pixel. and does not serialize writes to different pixels.

3.2 OpenMPI

Coordinator

```

1 void mpi_coordinator(const Arguments& args) {
2     mpi::coordinator_broadcast(args, mpi::Tag::Arguments);
3
4     u32 frame = 0;
5     u32 outstanding = mpi::worker_count();
6
7     while (frame < args.frames || outstanding!=0) {
8         auto receiver = mpi::Receiver::probe();
9
10        switch (receiver.type()) {
11            case mpi::Tag::FrameComplete: {
12                const auto finished_frame = receiver.receive<u32>();
13                if (finished_frame == 0xFFFFFFFF) {
14                    outstanding--;
15                }
16            }break;
17            case mpi::Tag::WorkerReady:
18                if (frame < args.frames) {
19                    receiver.receive<mpi::Empty>(nullptr).sender(mpi::Tag::BeginFrame).send(frame);
20                    frame++;
21                }else {
22                    receiver.receive<mpi::Empty>(nullptr).sender(mpi::Tag::BeginFrame).send(0xFFFFFFFF);
23                }break;
24            case mpi::Tag::LoadFile: {
25                auto path = receiver.receive<std::string>();
26                auto data = file_load_string(path);
27                receiver.sender().send(data);
28            }break;
29            default:break;
30        }
31    }
32 }
33 }
```

Worker

```

1 void mpi_worker() {
2     const auto args = mpi::Receiver::begin(mpi::COORDINATOR, mpi::Tag::Arguments).receive<Arguments>();
3     auto game = Game::make_game(args);
4
5     while (true) {
6         auto receiver = mpi::Sender::begin(mpi::COORDINATOR, mpi::Tag::WorkerReady).send(mpi::Empty{}).receiver(
7             mpi::Tag::BeginFrame);
8         u32 frame = receiver.receive<u32>();
9         if (frame == 0xFFFFFFFF) {
10             receiver.sender(mpi::Tag::FrameComplete).send(frame);
11             return;
12         }
13         game->render();
14     }
15 }
```

A custom typesafe wrapper around MPI calls was designed to help make development easier. Send and Receive wrap MPI_Recv and MPI_Send and provide the corresponding MPI type, count, and tag.

The architecture is a single coordinator and many worker model where the coordinator

coordinates which worker renders which frame and is also responsible for serving requested files.

The worker is responsible for rendering requested frames and also requesting any resources (textures, model data) if needed from the coordinator.

4 Performance Testing

Table 1: Performance Bench Parameters

Resolution	360x240, 720x480, 1080x1920
Scenes/Triangles	Wavy(7212), Halo(42600), Bricks(752456)
Parallelism	Single Thread, MP/MPI(#1, #2, #4, #8, #16)
Frames	300
CPU	AMD Ryzen 9 7950X (32) @ 5.883GHz
OS	NixOS 25.05 (Warbler) x86_64
RAM	64GiB DDR5
Compiler	clang version 19

4.1 Reproducibility

each scene had 300 different frames rendered to ensure random variations in time were minimal.

5 Results

5.1 Rendering Only

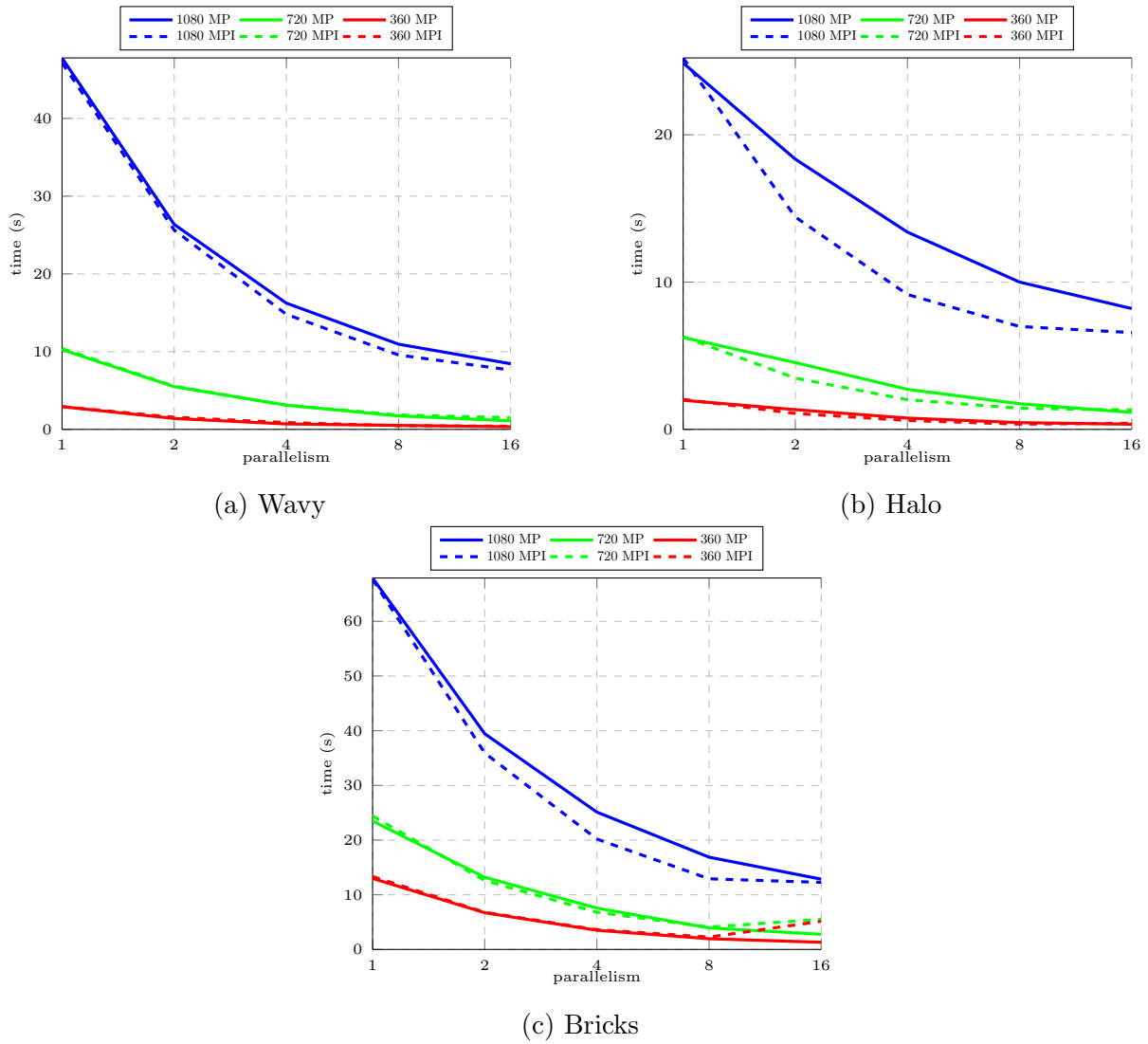


Figure 1: Scene Render Times

	2	4	8	16
Bricks 360	$S = 1.93\times$ $E = 96.70\%$	$S = 3.70\times$ $E = 92.49\%$	$S = 6.63\times$ $E = 82.81\%$	$S = 9.87\times$ $E = 61.71\%$
Bricks 720	$S = 1.78\times$ $E = 88.98\%$	$S = 3.11\times$ $E = 77.63\%$	$S = 5.96\times$ $E = 74.54\%$	$S = 8.47\times$ $E = 52.96\%$
Bricks 1080	$S = 1.72\times$ $E = 86.11\%$	$S = 2.70\times$ $E = 67.62\%$	$S = 4.03\times$ $E = 50.32\%$	$S = 5.28\times$ $E = 33.02\%$
Halo 360	$S = 1.48\times$ $E = 73.80\%$	$S = 2.57\times$ $E = 64.33\%$	$S = 4.31\times$ $E = 53.83\%$	$S = 5.66\times$ $E = 35.39\%$
Halo 720	$S = 1.38\times$ $E = 68.86\%$	$S = 2.30\times$ $E = 57.41\%$	$S = 3.59\times$ $E = 44.84\%$	$S = 5.43\times$ $E = 33.95\%$
Halo 1080	$S = 1.36\times$ $E = 67.81\%$	$S = 1.86\times$ $E = 46.44\%$	$S = 2.49\times$ $E = 31.09\%$	$S = 3.03\times$ $E = 18.96\%$
Wavy 360	$S = 2.08\times$ $E = 103.86\%$	$S = 4.18\times$ $E = 104.59\%$	$S = 5.69\times$ $E = 71.17\%$	$S = 8.67\times$ $E = 54.20\%$
Wavy 720	$S = 1.87\times$ $E = 93.40\%$	$S = 3.29\times$ $E = 82.34\%$	$S = 5.92\times$ $E = 74.02\%$	$S = 9.32\times$ $E = 58.25\%$
Wavy 1080	$S = 1.81\times$ $E = 90.62\%$	$S = 2.94\times$ $E = 73.56\%$	$S = 4.36\times$ $E = 54.50\%$	$S = 5.66\times$ $E = 35.36\%$
Average	$S = 1.71\times$ $E = 85.57\%$	$S = 2.96\times$ $E = 74.05\%$	$S = 4.77\times$ $E = 59.68\%$	$S = 6.82\times$ $E = 42.64\%$

Figure 2: MP Render Time

	2	4	8	16
Bricks 360	$S = 1.96\times$ $E = 98.00\%$	$S = 3.68\times$ $E = 92.08\%$	$S = 5.89\times$ $E = 73.63\%$	$S = 2.59\times$ $E = 16.17\%$
Bricks 720	$S = 1.94\times$ $E = 96.85\%$	$S = 3.58\times$ $E = 89.62\%$	$S = 5.96\times$ $E = 74.47\%$	$S = 4.44\times$ $E = 27.73\%$
Bricks 1080	$S = 1.88\times$ $E = 94.18\%$	$S = 3.35\times$ $E = 83.79\%$	$S = 5.24\times$ $E = 65.46\%$	$S = 5.52\times$ $E = 34.48\%$
Halo 360	$S = 1.89\times$ $E = 94.45\%$	$S = 3.38\times$ $E = 84.52\%$	$S = 5.75\times$ $E = 71.82\%$	$S = 4.89\times$ $E = 30.57\%$
Halo 720	$S = 1.81\times$ $E = 90.34\%$	$S = 3.12\times$ $E = 78.00\%$	$S = 4.39\times$ $E = 54.91\%$	$S = 4.77\times$ $E = 29.82\%$
Halo 1080	$S = 1.75\times$ $E = 87.47\%$	$S = 2.75\times$ $E = 68.86\%$	$S = 3.61\times$ $E = 45.14\%$	$S = 3.84\times$ $E = 23.98\%$
Wavy 360	$S = 1.88\times$ $E = 93.81\%$	$S = 3.35\times$ $E = 83.64\%$	$S = 6.19\times$ $E = 77.39\%$	$S = 7.17\times$ $E = 44.80\%$
Wavy 720	$S = 1.87\times$ $E = 93.50\%$	$S = 3.35\times$ $E = 83.64\%$	$S = 5.59\times$ $E = 69.90\%$	$S = 6.90\times$ $E = 43.10\%$
Wavy 1080	$S = 1.84\times$ $E = 91.96\%$	$S = 3.19\times$ $E = 79.63\%$	$S = 4.93\times$ $E = 61.58\%$	$S = 6.17\times$ $E = 38.59\%$
Average	$S = 1.87\times$ $E = 93.40\%$	$S = 3.31\times$ $E = 82.64\%$	$S = 5.28\times$ $E = 66.03\%$	$S = 5.14\times$ $E = 32.14\%$

Figure 3: MPI Render Time

5.2 Rendering And Resource Loading

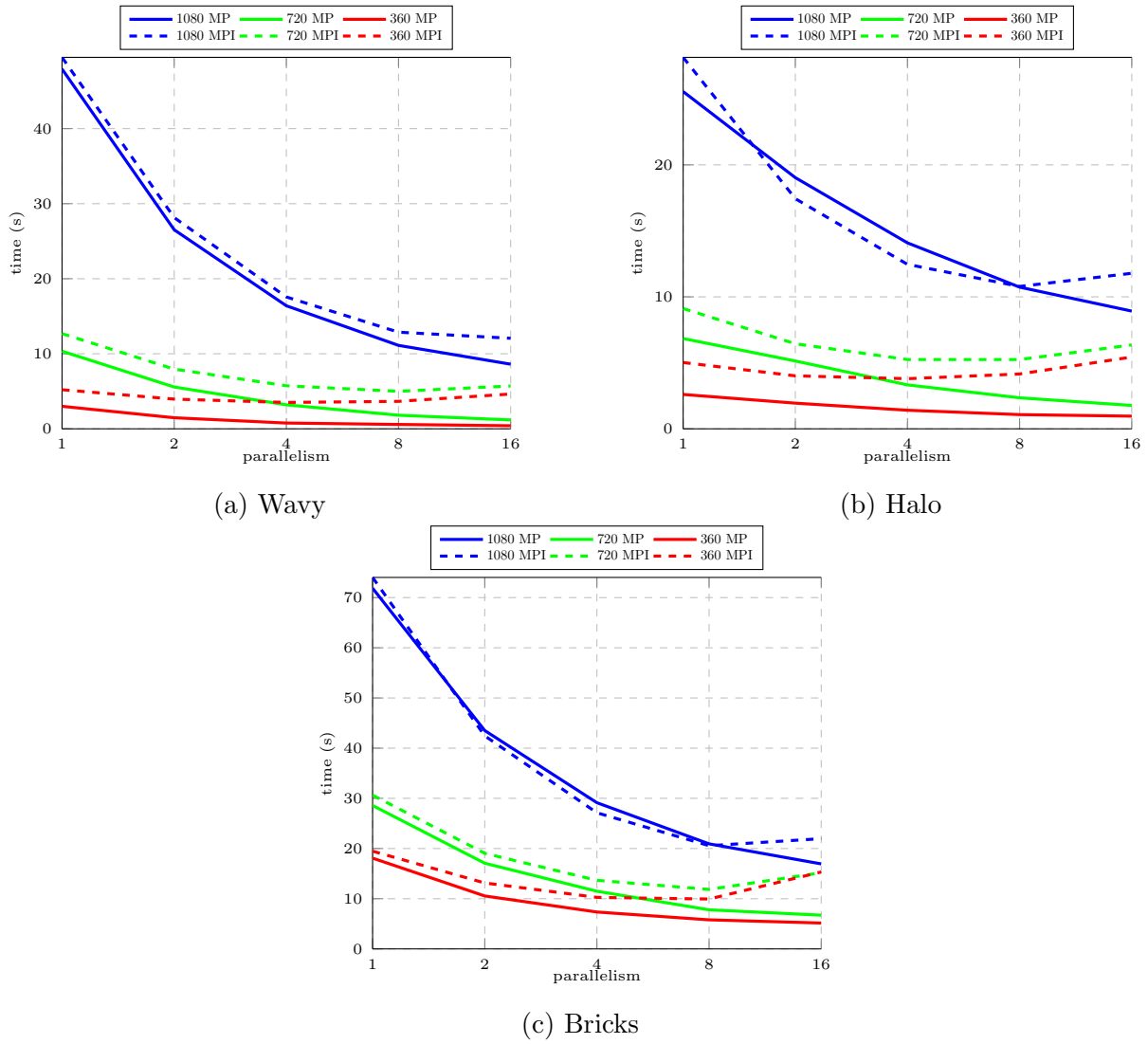


Figure 4: Scene Total Times

	2	4	8	16
Bricks 360	$S = 1.71\times$ $E = 85.71\%$	$S = 2.46\times$ $E = 61.50\%$	$S = 3.12\times$ $E = 39.04\%$	$S = 3.51\times$ $E = 21.92\%$
Bricks 720	$S = 1.67\times$ $E = 83.69\%$	$S = 2.49\times$ $E = 62.15\%$	$S = 3.67\times$ $E = 45.85\%$	$S = 4.25\times$ $E = 26.57\%$
Bricks 1080	$S = 1.65\times$ $E = 82.59\%$	$S = 2.47\times$ $E = 61.68\%$	$S = 3.43\times$ $E = 42.92\%$	$S = 4.24\times$ $E = 26.52\%$
Halo 360	$S = 1.34\times$ $E = 66.94\%$	$S = 1.85\times$ $E = 46.25\%$	$S = 2.42\times$ $E = 30.23\%$	$S = 2.72\times$ $E = 16.97\%$
Halo 720	$S = 1.33\times$ $E = 66.54\%$	$S = 2.05\times$ $E = 51.37\%$	$S = 2.91\times$ $E = 36.37\%$	$S = 3.86\times$ $E = 24.14\%$
Halo 1080	$S = 1.34\times$ $E = 67.13\%$	$S = 1.81\times$ $E = 45.34\%$	$S = 2.38\times$ $E = 29.76\%$	$S = 2.86\times$ $E = 17.90\%$
Wavy 360	$S = 2.03\times$ $E = 101.52\%$	$S = 3.90\times$ $E = 97.47\%$	$S = 5.14\times$ $E = 64.23\%$	$S = 7.33\times$ $E = 45.81\%$
Wavy 720	$S = 1.86\times$ $E = 92.80\%$	$S = 3.24\times$ $E = 80.90\%$	$S = 5.70\times$ $E = 71.28\%$	$S = 8.74\times$ $E = 54.65\%$
Wavy 1080	$S = 1.81\times$ $E = 90.38\%$	$S = 2.92\times$ $E = 73.09\%$	$S = 4.31\times$ $E = 53.91\%$	$S = 5.57\times$ $E = 34.80\%$
Average	$S = 1.64\times$ $E = 81.92\%$	$S = 2.58\times$ $E = 64.42\%$	$S = 3.68\times$ $E = 45.95\%$	$S = 4.79\times$ $E = 29.92\%$

Figure 5: MP Total Time

	2	4	8	16
Bricks 360	$S = 1.48\times$ $E = 74.12\%$	$S = 1.90\times$ $E = 47.45\%$	$S = 1.96\times$ $E = 24.49\%$	$S = 1.27\times$ $E = 7.94\%$
Bricks 720	$S = 1.61\times$ $E = 80.42\%$	$S = 2.24\times$ $E = 56.06\%$	$S = 2.58\times$ $E = 32.31\%$	$S = 2.02\times$ $E = 12.62\%$
Bricks 1080	$S = 1.74\times$ $E = 87.18\%$	$S = 2.73\times$ $E = 68.23\%$	$S = 3.60\times$ $E = 44.99\%$	$S = 3.37\times$ $E = 21.06\%$
Halo 360	$S = 1.25\times$ $E = 62.59\%$	$S = 1.32\times$ $E = 33.06\%$	$S = 1.21\times$ $E = 15.11\%$	$S = 0.92\times$ $E = 5.77\%$
Halo 720	$S = 1.41\times$ $E = 70.71\%$	$S = 1.74\times$ $E = 43.40\%$	$S = 1.74\times$ $E = 21.71\%$	$S = 1.43\times$ $E = 8.96\%$
Halo 1080	$S = 1.61\times$ $E = 80.69\%$	$S = 2.26\times$ $E = 56.47\%$	$S = 2.61\times$ $E = 32.61\%$	$S = 2.39\times$ $E = 14.93\%$
Wavy 360	$S = 1.32\times$ $E = 65.79\%$	$S = 1.48\times$ $E = 37.04\%$	$S = 1.42\times$ $E = 17.78\%$	$S = 1.12\times$ $E = 7.02\%$
Wavy 720	$S = 1.60\times$ $E = 79.76\%$	$S = 2.21\times$ $E = 55.28\%$	$S = 2.54\times$ $E = 31.75\%$	$S = 2.22\times$ $E = 13.89\%$
Wavy 1080	$S = 1.76\times$ $E = 87.94\%$	$S = 2.82\times$ $E = 70.44\%$	$S = 3.84\times$ $E = 48.06\%$	$S = 4.10\times$ $E = 25.65\%$
Average	$S = 1.53\times$ $E = 76.58\%$	$S = 2.08\times$ $E = 51.94\%$	$S = 2.39\times$ $E = 29.87\%$	$S = 2.09\times$ $E = 13.09\%$

Figure 6: MPI Total Time

6 Analysis

Time was measured in two ways, "Render Time" which only accounts for the time it takes to render frames. and "Total Time" which accounts for asset loading and resource sharing.

Both times were recorded from the exact same runs so the difference is entirely the extra time taken for resource loading.

Also importantly, MPI and MP parallelize different parts of the program with differing levels of granularity.

6.1 Render Time Only

The graphs which show these are 4, 5, and 6. Interestingly (and unexpectedly) the MPI version performs the same if not better than the MP version in almost all cases. This really caught me off guard when I first looked at the results (so much so I double checked I had the correct data points). But I think a reasonable explanation for this performance difference is true sharing(Cache Coherence). It cannot be false sharing because each pixel is larger than 64 bytes (the cache line on the benchmarking system) and is aligned to 128 byte boundaries. However, in the MP version each thread is rendering over the same image. This means that the L1 cache in particular (for each core) must sync its changes with other cores if they request the same address (to ensure cache coherence). In contrary the MPI implementation has each node(thread in our case) work in its own image. this means that each thread uses its own frame buffer and therefore does not have to constantly sync its L1 cache with other cores when rendering.

Even so, once the degree of parallelism reaches 16 the overhead of MPI can be seen as the curves of MPI and MP begin to intersect again.

Other than that the graphs and data is about what you'd expect. Both version have almost the exact same performance to begin with. Efficiency drops as parallelism increases. And the effective speed up begins to slow.

6.2 Total Time

These graphs 1, and tables 2 3 show the total time benchmarks and performance.

The MPI versions are almost always slower, with the only exception being in a few of the higher resolution tests.

This is due to the overhead of sending/parsing the assets and textures n times where n is the degree of parallelism. Even in the cases where it has runs which are faster at first the

performance quickly becomes dominated by the startup time of duplicated work and large data transfers.

In both cases the efficiency decreases much quicker compared to render time only which makes sense as the quicker our render time is, the most we are benchmarking "how fast can we read and parse images and model data".

6.3 Further Testing

I did not include this in my report (as I do not have an environment to test it in effectively). But MPI and MP can be used together with this program. Since they parallelize different sections of the code it is possible to compile the program with both libraries/compiler options enabled (I also tested it to make sure it worked). This could be useful as each MPI node could take full advantage of all local CPU cores on its machine to limit bandwidth/re computation of loading and parsing assets. Additionally each MPI node would only need to request the assets it needs, which if different frames only need certain assets it could be significant savings in communication overhead.

7 Building and Running

Two separate build files are provided for each respective compiler `build_clang.sh` and `build_gcc.sh`.

Important: to switch between building with gcc or clang you **must** completely delete all `cmake-build-*` folders if it has been created previously with the other build script.

Also if MP, MPI, or OpenGL is not installed on the build system. that respective build will fail. It will not effect the other builds even if an error is shown when building.

Once the project has been built you can perform the same benchmarks in this document by running `bench.sh`. It will run through all the permutations of the input parameters.

Neither the halo nor bricks model is included in the submissions due to licensing reasons though the wavy scene will work just fine. If you wish to run them you must download the models [5] [6] as OBJ files and place the extracted model in the `assets` folder named `halo` and `bricks` respectively.

References

- [1] S. Barrett, “stb image.” [Online]. Available: <https://github.com/nothings/stb>
- [2] —, “stb image write.” [Online]. Available: <https://github.com/nothings/stb>
- [3] S. Fujita, “Tiny obj loader.” [Online]. Available: <https://github.com/tinyobjloader/tinyobjloader>
- [4] “Learn opengl.” [Online]. Available: <https://learnopengl.com/>
- [5] “Halo model.” [Online]. Available: <https://sketchfab.com/3d-models/spartan-armour-mkv-halo-reach-57070b2fd9ff472c8988e76d8c5cbe66>
- [6] “Bricks model.” [Online]. Available: <https://sketchfab.com/3d-models/ziegelpfeiler-05ea628cfcea4cbe9837038ef49cd42a>